```
<%= turbo_stream.replace(
    "subtotal",
    partial: "venues/subtotal",
    locals: {concert: @concert, user: @user}
  ) %>
```

Receiving Commands via ActionCable

The interactions work now, but we still need to be able to catch the ActionCable broadcast if another browser on the same page holds a ticket. The tricky part here is that the display depends on a piece of client-side state—the "tickets-to-buy count" number—that the server won't know when it makes the broadcast.

I thought of three ways this might work:

- We could make the "tickets-to-buy count" number server-side state by having the server store it in a session or something and also broadcast that information to other browsers via ActionCable when it changes. I decided this would be confusing for our mythical users.

- We could have the ActionCable broadcast send versions of the seat grid with all possible ticket-to-buy numbers and have the receiver pick the correct one for display. I considered this one pretty seriously, and I think it might be a good choice in another context, but in this case I thought the possible amount of extra data being sent was too high.

- We could have the ActionCable broadcast be received as a signal for the client to call the server and request the seat grid with the correct ticket-to-buy number, which is the option I eventually chose. There's an extra server call here, but overall I think this is the easiest solution to manage.

One reason I think the last version works for us is that the request the client would make—please give us the new seat grid and subtotal—is exactly the same request we're already making when the pull-down menu changes. We can leverage that. (This is why we added the subtotal call to the form response.)

We've already set this up in the markup: the pull-down form declares a Stimulus